

8-Bit Custom CPU — Complete Logisim Build Guide

Step-by-Step Manual Construction with Instruction Register (IR)

Harvard Architecture · 8-bit Datapath · 8-Instruction ISA · Circular Left Shift Program

0. What You Will Build

This guide walks you through building every sub-circuit manually in Logisim Evolution — no XML import, no copy-paste. By the end you will have a working CPU that continuously performs a circular left shift on a 10-element array stored in RAM.

Component	What it does	Key signals
Program ROM	Stores the 13-instruction program	8-bit addr → 16-bit instruction
IR (Instruction Register)	Latches instruction on clock edge	16-bit D → 16-bit Q
Splitter / Decoder	Breaks instruction into fields	opcode[3:0], reg[1:0], imm[7:0]
PC + Adder	Tracks and advances program address	CLK, Jump, Branch → next addr
Control Unit ROM	Maps opcode to 8 control bits	4-bit opcode → 8-bit control word
Register File (R0–R3)	4 × 8-bit general-purpose registers	Write_Addr, RegWrite, CLK
ALU	ADD, SUB, CMP operations	In_A, In_B, ALU_op → result, ZF
Data RAM	Stores array a[0]–a[9]	Addr, DataIn, MemWr, MemRd
Write-back MUX	Selects what goes back to register	ImmReg, DataReg, ALU → reg

1. Before You Start

Required software

Logisim Evolution v4.1.0 or later — download from github.com/logisim-evolution

This version supports the ROM/RAM content editor and tunnel labels needed here.

Create a new project: File → New. Save immediately as `cpu_shift.circ`. You will work in the default circuit called `main`.

Create sub-circuits first. Go to Project → Add Circuit and create these four named circuits before touching the main canvas:

- Control_Unit
- Register_File
- ALU
- (main already exists — do not add it)

Naming rule

Tunnel labels are case-sensitive in Logisim Evolution. Copy every label exactly as written in this guide — one wrong character means a disconnected wire.

2. ISA & Control Word Reference (Keep This Open)

Instruction format: [Opcode 4 bits][Register 2 bits][Unused 2 bits][Immediate 8 bits] = 16 bits total

Opcode (bin)	Mnemonic	Jump	Branch	ImmReg	DataReg	ALU	MemRd	MemWr	RegWr	Hex
0000	LDI	0	0	1	0	0	0	0	1	0x21
0001	LD	0	0	0	1	0	1	0	1	0x15
0010	ST	0	0	0	0	0	0	1	0	0x02
0011	ADD	0	0	0	0	0	0	0	1	0x01
0100	SUB	0	0	0	0	1	0	0	1	0x09
0101	CMP	0	0	0	0	1	0	0	0	0x08
0110	JNE	0	1	0	0	0	0	0	0	0x40
0111	JMP	1	0	0	0	0	0	0	0	0x80

Program ROM contents (enter exactly in ROM editor):

```
|addr/data: 8 16 c00 1400 800 3c01 1000 4c01 2000 3801 3c01 5809 6004 2400 7000
```

Fix applied

Address 0x0A is 6004 (JNE jumps to 04, not 03). This fixes the pointer/counter desync bug. See analysis for full explanation.

PART A — Build the Sub-Circuits

3. Build the ALU Sub-Circuit

Double-click **ALU** in the project explorer to open it. Everything below goes inside this circuit.

Step 1 Place input pin: In_A (8-bit)

- Select: Wiring → Pin from the left toolbar
- Click to place it on the canvas, roughly left-centre
- Right-click it → Edit Properties: Label = In_A, Data Bits = 8, Output? = No (it is an input)

Step 2 Place input pin: In_B (8-bit)

- Place another Pin below In_A
- Label = In_B, Data Bits = 8, Output? = No

Step 3 Place input pin: ALU_op (1-bit)

- Place a Pin to the right of both inputs
- Label = ALU_op, Data Bits = 1, Output? = No
- This 1-bit signal selects between Adder (0) and Subtractor (1)

Step 4 Place the Adder

- Select: Arithmetic → Adder
- Place it to the right of In_A and In_B
- Properties: Data Bits = 8
- Wire In_A top pin → Adder top input (cin can be left unconnected, it defaults to 0)
- Wire In_B top pin → Adder bottom input

Step 5 Place the Subtractor

- Select: Arithmetic → Subtractor
- Place it below the Adder, roughly same X position
- Properties: Data Bits = 8
- Wire In_A → Subtractor top input (branch the wire from In_A downward)
- Wire In_B → Subtractor bottom input (branch the wire from In_B downward)

Step 6 Place the result MUX (selects Adder vs Subtractor)

- a. Select: Plexers → Multiplexer
- b. Place it to the right of both arithmetic units
- c. Properties: Data Bits = 8, Select Bits = 1
- d. Wire Adder output → MUX input 0 (top input, sel=0 = ADD)
- e. Wire Subtractor output → MUX input 1 (bottom input, sel=1 = SUB/CMP)
- f. Wire ALU_op pin → MUX select pin (the small pin at the bottom of the MUX)

Step 7 Place output pin: ALU_Out (8-bit)

- a. Place a Pin to the right of the MUX
- b. Label = ALU_Out, Data Bits = 8, Output? = Yes
- c. Wire MUX output → ALU_Out pin

Step 8 Build the Zero Flag (ZF) — 8-input NOR gate

- a. Select: Gates → NOR Gate
- b. Properties: Number of Inputs = 8
- c. Place it below the MUX
- d. The 8 inputs of the NOR gate must each connect to one bit of the MUX output
- e. To do this: place a Splitter (Wiring → Splitter) between the MUX output and the NOR gate
- f. Splitter properties: Facing = East, Fan-Out = 8, Incoming = 8 (splits 8-bit bus into 8 single bits)
- g. Wire MUX output → Splitter combined end (left side)
- h. Wire each Splitter individual output → one NOR gate input
- i. If ZF=1 the ALU result is zero (meaning CMP found equality)

Step 9 Place output pin: Zero_Flag (1-bit)

- a. Place a Pin to the right of the NOR gate
- b. Label = Zero_Flag, Data Bits = 1, Output? = Yes
- c. Wire NOR gate output → Zero_Flag pin

ALU complete — verify before continuing

In_A and In_B feed both Adder and Subtractor in parallel.
ALU_op=0 → MUX passes Adder result to ALU_Out.
ALU_op=1 → MUX passes Subtractor result to ALU_Out.
The NOR of all 8 output bits gives Zero_Flag=1 when result=0.

4. Build the Register_File Sub-Circuit

Double-click **Register_File** in the project explorer.

Step 10 Place all input pins

- a. Pin: Label=RegWrite, Bits=1, Output=No — top left area
- b. Pin: Label=Write_Addr, Bits=2, Output=No — below RegWrite
- c. Pin: Label=Data_In, Bits=8, Output=No — below Write_Addr
- d. Pin: Label=Clock, Bits=1, Output=No — bottom left
- e. Pin: Label=Read_Addr, Bits=2, Output=No — below Clock

Step 11 Place the write decoder (2-to-4 Decoder)

- a. Select: Plexers → Decoder
- b. Properties: Select Bits = 2 (gives 4 outputs), Facing = South
- c. Place it in the upper-middle area
- d. Wire Write_Addr → Decoder select input
- e. The decoder outputs 4 individual enable signals (one per register)

Step 12 Place the 4 registers (R0, R1, R2, R3)

- a. Select: Memory → Register
- b. Properties: Data Bits = 8, Trigger = Rising Edge, Appearance = Logisim Evolution
- c. Place R0 below the decoder, leftmost
- d. Place R1 to the right of R0, same height
- e. Place R2 to the right of R1
- f. Place R3 to the right of R2
- g. Label each register by right-clicking → Edit Properties → Label: R0, R1, R2, R3

Step 13 Connect write-enable to each register

- a. The Decoder has 4 output pins (D0, D1, D2, D3) — one per register
- b. Each decoder output must be ANDed with RegWrite before going to a register's enable pin
- c. Place 4 AND Gates (Gates → AND Gate, 2 inputs each)
- d. Wire Decoder output 0 → AND gate 0 input A; Wire RegWrite → AND gate 0 input B
- e. Repeat for gates 1, 2, 3 with Decoder outputs 1, 2, 3
- f. Wire AND gate 0 output → R0 enable pin
- g. Wire AND gate 1 output → R1 enable pin
- h. Wire AND gate 2 output → R2 enable pin
- i. Wire AND gate 3 output → R3 enable pin

Step 14 Connect Data_In and Clock to all registers

- a. Branch the Data_In wire horizontally across all 4 registers
- b. Connect the branch to every register's D (data input) pin
- c. Branch the Clock wire horizontally across all 4 registers
- d. Connect the branch to every register's CLK pin

Step 15 Build the read MUX (4-to-1 for register output)

- a. Select: Plexers → Multiplexer
- b. Properties: Data Bits = 8, Select Bits = 2, Facing = South
- c. Place it below the 4 registers, centred
- d. Wire R0 output → MUX input 0
- e. Wire R1 output → MUX input 1
- f. Wire R2 output → MUX input 2
- g. Wire R3 output → MUX input 3
- h. Wire Read_Addr → MUX select pin

Step 16**Place output pins**

- a. Pin: Label=Reg_Out, Bits=8, Output=Yes — wire from MUX output
- b. Pin: Label=R3_Pointer, Bits=8, Output=Yes — wire from R3 register output directly (this gives the ALU and RAM the memory pointer)

Register File complete — verify

Write path: Write_Addr → Decoder → AND(RegWrite) → each register enable.

Data and Clock fan out to all 4 registers simultaneously.

Read path: Read_Addr → MUX select → Reg_Out.

R3_Pointer always shows R3 regardless of read address.

5. Build the Control_Unit Sub-Circuit

Double-click **Control_Unit** in the project explorer.

Step 17 Place input pin: Opcode (4-bit)

- Pin: Label=Opcode, Data Bits=4, Output=No
- Place on the left side of the canvas

Step 18 Place the Control ROM

- Select: Memory → ROM
- Properties: Address Bit Width = 4 (16 possible opcodes), Data Bit Width = 8
- Label it: right-click → Edit Properties → Label = Control ROM
- Wire Opcode input pin → ROM address input
- Right-click the ROM → Edit Contents. Enter exactly:

addr/data: 4 8 21 15 02 01 09 08 40 80

Step 19 Split the 8-bit control word into individual signals

- Select: Wiring → Splitter
- Properties: Facing = East, Fan-Out = 8, Incoming = 8
- Each individual output of the splitter is one control bit
- Place the splitter to the right of the ROM, wire ROM data output → splitter combined input
- The bit mapping from the splitter outputs (bit 0 = rightmost = RegWr):

Splitter bit	Control signal	Active when
bit 7 (MSB)	Jump	JMP instruction
bit 6	Branch	JNE instruction
bit 5	ImmReg	LDI — write immediate to register
bit 4	DataReg	LD — write RAM data to register
bit 3	ALU_op	SUB or CMP — selects subtractor
bit 2	MemRd	LD — enable RAM read
bit 1	MemWr	ST — enable RAM write
bit 0 (LSB)	RegWr	Any instruction that writes a register

Step 20 Place 8 output pins — one per control signal

- Place 8 output Pins on the right side of the canvas
- From top to bottom: RegWrite, MemWrite, MemRead, ALU_op, DataReg, ImmReg, Branch, Jump
- Wire splitter bit 0 → RegWrite output pin
- Wire splitter bit 1 → MemWrite output pin
- Wire splitter bit 2 → MemRead output pin
- Wire splitter bit 3 → ALU_op output pin

- g. Wire splitter bit 4 → DataReg output pin
- h. Wire splitter bit 5 → ImmReg output pin
- i. Wire splitter bit 6 → Branch output pin
- j. Wire splitter bit 7 → Jump output pin

Critical naming — copy exactly

Output pin labels must match tunnel names in the main circuit.

Use: RegWrite MemWrite MemRead ALU_op DataReg ImmReg Branch Jump

Do NOT use underscores like Mem_Write — this breaks tunnel connections.

PART B — Build the Main Circuit

Now open the main circuit. Everything from here onwards goes into main. Leave plenty of space — this circuit is large. Work left to right: Fetch → Decode → Execute → Write-back.

6. Program Counter (PC)

Step 21 Place the PC register

- a. Select: Memory → Register
- b. Properties: Data Bits = 8, Trigger = Rising Edge
- c. Label = PC
- d. Place on the far left of the canvas

Step 22 Place the PC increment adder

- a. Select: Arithmetic → Adder
- b. Properties: Data Bits = 8
- c. Place above and to the right of the PC register
- d. Wire PC output → Adder input A
- e. Place a Constant: Wiring → Constant, value = 1, bits = 8
- f. Wire Constant (1) → Adder input B
- g. Adder output = PC + 1 = next address

Step 23 Place the Clock

- a. Select: Wiring → Clock
- b. Place near the top left
- c. Add a Tunnel: Wiring → Tunnel, Label = CLK, Data Bits = 1
- d. Wire Clock output → CLK tunnel
- e. Add another CLK tunnel near the PC register and wire it to the PC clock pin
- f. You will add more CLK tunnels throughout the circuit — they all connect automatically via label

Step 24 Place the PC input MUX (selects next PC value)

- a. Select: Plexers → Multiplexer
- b. Properties: Data Bits = 8, Select Bits = 1
- c. Place between the adder and the PC register input
- d. MUX input 0 = PC + 1 (normal execution) — wire from adder output
- e. MUX input 1 = Jump target address (from instruction immediate field)
- f. Add a Tunnel: Label = Imm, Data Bits = 8. Wire it to MUX input 1

- g. Add a Tunnel: Label = sel, Data Bits = 1. Wire it to MUX select pin
- h. Wire MUX output → PC register D input

7. Program ROM (Instruction Memory)

Step 25 Place the program ROM

- a. Select: Memory → ROM
- b. Properties: Address Bit Width = 8, Data Bit Width = 16
- c. Place to the right of the PC
- d. Wire PC output → ROM address input
- e. Right-click ROM → Edit Contents. Enter the program:

addr/data: 8 16 c00 1400 800 3c01 1000 4c01 2000 3801 3c01 5809 6004 2400 7000

8. Instruction Register (IR) — NEW

This is the missing component from the original circuit. It latches the 16-bit instruction from the ROM on every rising clock edge, holding it stable throughout the entire execution cycle.

Why this is important

Without IR: ROM output changes the moment PC changes — instruction can shift mid-cycle.

With IR: instruction is frozen in the register for the full clock period.

This is architecturally correct for a single-cycle CPU.

Step 26 Place the IR register

- a. Select: Memory → Register
- b. Properties: Data Bits = 16, Trigger = Rising Edge
- c. Label = IR
- d. Place it directly to the right of the program ROM

Step 27 Connect IR to ROM and Clock

- a. Wire ROM data output (16-bit) → IR register D input
- b. Add a CLK tunnel near the IR and wire it to IR clock pin
- c. The IR Q output (16-bit) is now the instruction that feeds the rest of the circuit

Step 28 Connect IR output to the instruction splitter

- a. Select: Wiring → Splitter
- b. Properties: Facing = East, Incoming = 16, Fan-Out = 4
- c. Set bit assignments in splitter properties:
- d. Bits 7–0 → Fan-out group 0 (this is the 8-bit Immediate field)
- e. Bits 9–8 → Fan-out group 1 (this is the 2-bit Register Address field)
- f. Bits 11–10 → set to None (unused bits — no output)
- g. Bits 15–12 → Fan-out group 3 (this is the 4-bit Opcode field)
- h. Place the splitter to the right of the IR
- i. Wire IR Q output → Splitter combined input (left pin)

Step 29 **Add tunnels for the decoded fields**

- a. From Splitter fan-out 0 (Imm): add Tunnel Label=Imm, Bits=8
- b. From Splitter fan-out 1 (Reg): add Tunnel Label=Reg_Addr, Bits=2
- c. From Splitter fan-out 3 (Opcode): add Tunnel Label=opcode, Bits=4
- d. These tunnels carry signals to the Control Unit and Register File automatically

9. Place the Control Unit in Main

Step 30 **Place the Control_Unit component**

- a. In the left panel under your project circuits, find Control_Unit
- b. Drag it onto the main canvas, roughly in the upper-centre area
- c. The component shows pins matching what you defined inside it

Step 31 **Connect opcode to Control Unit**

- a. Add a Tunnel: Label=opcode, Bits=4 near the Control Unit input
- b. Wire the tunnel to the Opcode input pin of the Control Unit component

Step 32 **Add output tunnels for all control signals**

- a. Wire Control Unit RegWrite output → Tunnel Label=RegWrite, Bits=1
- b. Wire Control Unit MemWrite output → Tunnel Label=MemWrite, Bits=1
- c. Wire Control Unit MemRead output → Tunnel Label=MemRead, Bits=1
- d. Wire Control Unit ALU_op output → Tunnel Label=a_op, Bits=1
- e. Wire Control Unit DataReg output → (used for W_MUX select, see step 43)
- f. Wire Control Unit ImmReg output → (used for W_MUX select, see step 43)
- g. Wire Control Unit Branch output → Tunnel Label=Branch, Bits=1
- h. Wire Control Unit Jump output → Tunnel Label=Jump, Bits=1

10. Place the Register File in Main

Step 33 **Place Register_File component**

- a. Drag Register_File from the project panel onto main canvas, centre area

Step 34 **Connect all inputs to Register File**

- a. Wire Tunnel RegWrite → Register_File RegWrite pin
- b. Wire Tunnel Reg_Addr → Register_File Write_Addr pin
- c. Wire Tunnel Reg_Addr → Register_File Read_Addr pin (same 2-bit address for both — the target register is both read and written)
- d. Add a CLK tunnel near Register_File and wire to its Clock pin
- e. Wire Register_File Reg_Out → Tunnel Label=Reg_Out, Bits=8
- f. Wire Register_File R3_Pointer → Tunnel Label=R3_pointer, Bits=8

Step 35 **Connect the write-back data input to Register File**

- a. The Data_In of the Register File comes from the write-back MUX (built in step 43)
- b. For now, add a Tunnel Label=W_MUX, Bits=8 and wire it to Register_File Data_In pin

c. You will complete this connection after building the write-back MUX

11. Place the ALU in Main

Step 36 Place ALU component

- a. Drag ALU from the project panel onto the main canvas, below the Register File

Step 37 Connect ALU inputs

- a. Wire Tunnel Reg_Out → ALU In_A pin (this is the register value being operated on)
- b. Wire Tunnel Imm → ALU In_B pin (the immediate value from the instruction)
- c. Wire Tunnel a_op → ALU ALU_op pin
- d. Wire ALU ALU_Out → Tunnel Label=r_op, Bits=8
- e. Wire ALU Zero_Flag → Tunnel Label=ZF, Bits=1

12. Data RAM (Array Storage)

Step 38 Place the RAM component

- a. Select: Memory → RAM
- b. Properties: Address Bit Width = 8, Data Bit Width = 8, Trigger = Falling Edge
- c. Falling edge trigger ensures write happens after the read in the same cycle
- d. Place RAM on the right side of the canvas

Step 39 Connect RAM address and data

- a. Wire Tunnel R3_pointer → RAM address input (R3 always holds the memory address)
- b. Wire Tunnel Reg_Out → RAM data input (register value to store)
- c. Wire Tunnel MemWrite → RAM write-enable pin (WE)
- d. Wire Tunnel MemRead → RAM output-enable pin (OE)
- e. Add a CLK tunnel near RAM and wire to RAM clock pin
- f. Wire RAM data output → Tunnel Label=W_MUX (this will be one of the write-back MUX inputs)

Step 40 Load initial array contents into RAM

- a. Right-click RAM → Edit Contents
- b. Enter the 10 array values at addresses 00 through 09:

00 01 02 03 04 05 06 07 08 09

Demo tip

During the demo, load a different array here before starting the clock. After 10 clock cycles you should see the array shift left by one position with the original a[0] appearing at a[9].

13. Write-Back MUX

This MUX chooses what data gets written back into a register after an instruction executes.

sel[1:0]	DataReg bit	ImmReg bit	Source selected
00	0	0	ALU result (ADD, SUB, CMP)
01	0	1	Immediate value (LDI)
10	1	0	RAM data output (LD)
11	1	1	Unused / tie to 0

Step 41 Build the 2-bit select signal for W_MUX

- The Control Unit outputs DataReg (bit 1) and ImmReg (bit 0) as separate 1-bit wires
- You need to combine them into a 2-bit bus for the MUX select
- Place a Splitter: Wiring → Splitter, Fan-Out = 2, Incoming = 2, Facing = West
- Wire ImmReg control signal → Splitter individual bit 0 input
- Wire DataReg control signal → Splitter individual bit 1 input
- Add Tunnel Label=wm_sel, Bits=2 on the Splitter combined output

Step 42 Place the Write-Back MUX

- Select: Plexers → Multiplexer
- Properties: Data Bits = 8, Select Bits = 2
- Place it between the ALU/RAM outputs and the Register File data input

Step 43 Connect Write-Back MUX inputs

- Wire Tunnel r_op (ALU result) → MUX input 0 (sel=00)
- Wire Tunnel Imm (sign-extended to 8 bits) → MUX input 1 (sel=01)
- Wire RAM data output → MUX input 2 (sel=10)
- Tie MUX input 3 to Constant 0 (sel=11 unused)
- Wire Tunnel wm_sel → MUX select pin
- Wire MUX output → Tunnel W_MUX, Bits=8 (this feeds Register_File Data_In)

14. Branch and Jump Logic (PC Control)

This logic decides whether the PC takes the jump target or the sequential next address.

Step 44 Build the JNE (branch if not zero) logic

- Place a NOT Gate: Gates → NOT Gate
- Wire Tunnel ZF → NOT Gate input
- NOT output = 1 when ZF=0 (result was NOT zero = not equal)
- Place an AND Gate: Gates → AND Gate, 2 inputs
- Wire NOT gate output → AND gate input A
- Wire Tunnel Branch → AND gate input B
- AND output = 1 only when Branch=1 AND ZF=0 → branch is taken

Step 45 Combine branch and jump into the PC select signal

- Place an OR Gate: Gates → OR Gate, 2 inputs
- Wire AND gate output → OR gate input A
- Wire Tunnel Jump → OR gate input B
- OR output = 1 when either a branch is taken or an unconditional jump occurs
- Wire OR gate output → Tunnel Label=sel, Bits=1
- This sel tunnel connects to the PC input MUX select you placed in step 24

No D flip-flop on the branch path

The original circuit had a D flip-flop between the AND gate and OR gate.

This caused a 1-cycle delay — the branch fired one instruction late.

Do NOT add a DFF here. The combinational path is correct for single-cycle design.

15. Add Debug Probes

Probes let you watch values during simulation without wiring to output pins.

Step 46

Add probes for demo visibility

- Select: Wiring → Probe
- Place a probe on the R3_pointer tunnel — label it R3, set Radix = Unsigned Decimal
- Place a probe on the RAM data output — label it RAM_out, Radix = Signed Decimal
- Place a probe on the W_MUX output — label it RegIn, Radix = Unsigned Decimal
- Place a probe on the PC output — label it PC, Radix = Unsigned Decimal
- Place a probe on the IR output — label it IR, Radix = Hex — this shows the current instruction word

16. Complete Connection Checklist

Before running simulation, verify every connection below. A missing wire is silent — the simulation runs but produces wrong output.

From	To	Signal	Width
Clock	CLK tunnels everywhere	CLK	1
PC output	Program ROM address	PC addr	8
Program ROM output	IR register D input	instruction	16
IR register Q output	Splitter input	instruction	16
Splitter group 3	Tunnel opcode	opcode	4
Splitter group 1	Tunnel Reg_Addr	reg addr	2
Splitter group 0	Tunnel Imm	immediate	8
Tunnel opcode	Control_Unit Opcode pin	opcode	4
Control_Unit RegWrite	Tunnel RegWrite	RegWrite	1
Control_Unit MemWrite	Tunnel MemWrite	MemWrite	1
Control_Unit MemRead	Tunnel MemRead	MemRead	1
Control_Unit ALU_op	Tunnel a_op	ALU op	1
Control_Unit Branch	Tunnel Branch	Branch	1
Control_Unit Jump	Tunnel Jump	Jump	1
Tunnel Reg_Addr	Register_File Write_Addr	write addr	2
Tunnel Reg_Addr	Register_File Read_Addr	read addr	2
Tunnel RegWrite	Register_File RegWrite	write enable	1
Tunnel W_MUX	Register_File Data_In	write data	8
CLK tunnel	Register_File Clock	clock	1
Register_File Reg_Out	Tunnel Reg_Out	reg value	8
Register_File R3_Pointer	Tunnel R3_pointer	mem addr	8
Tunnel Reg_Out	ALU In_A	operand A	8
Tunnel Imm	ALU In_B	operand B	8
Tunnel a_op	ALU ALU_op	op select	1
ALU ALU_Out	Tunnel r_op	ALU result	8
ALU Zero_Flag	Tunnel ZF	zero flag	1
Tunnel R3_pointer	RAM address	mem addr	8
Tunnel Reg_Out	RAM data in	write data	8
Tunnel MemWrite	RAM WE pin	write enable	1

From	To	Signal	Width
Tunnel MemRead	RAM OE pin	output enable	1
CLK tunnel	RAM clock pin	clock	1
Tunnel r_op	W_MUX input 0	ALU result	8
Tunnel Imm	W_MUX input 1	immediate	8
RAM data out	W_MUX input 2	RAM read	8
W_MUX output	Tunnel W_MUX	write back	8
PC+1 adder	PC MUX input 0	next addr	8
Tunnel Imm	PC MUX input 1	jump addr	8
Tunnel sel	PC MUX select	jump/branch	1
PC MUX output	PC register D	new PC	8

17. Running the Simulation

Step 47	Set simulation speed
a. Simulate → Simulation Frequency → set to 8 Hz (8 steps per second) b. This is slow enough to watch the array shift step by step	

Step 48	Reset and start
a. Simulate → Reset Simulation (Ctrl+R) b. Simulate → Auto-Tick Enabled (Ctrl+K) to start the clock c. Watch the PC probe increment each tick d. Watch the IR probe show each instruction word in sequence e. After 13 ticks the program wraps back to PC=00 and repeats	

Step 49	Verify the circular shift
a. After the first full program run, open RAM (right-click → Edit Contents) b. The array should have shifted: [1,2,3,4,5,6,7,8,9,0] c. After the second full run: [2,3,4,5,6,7,8,9,0,1] d. After 10 runs: back to [0,1,2,3,4,5,6,7,8,9] — full rotation	

Step 50	For demo — load professor's array
a. Stop the simulation: Simulate → Auto-Tick Enabled (toggle off) b. Reset: Ctrl+R c. Right-click RAM → Edit Contents → load the professor's array values d. Right-click PC register → Set Value → 0 (to reset program counter) e. Restart the clock f. Show the R3 probe to demonstrate which array location is being accessed	

18. Final Pre-Demo Checklist

Must-pass checks before demo

1. Program ROM has 6004 at address 0x0A (not 6003) — fixes loop pointer bug
2. Control ROM in Control_Unit has exactly: 21 15 02 01 09 08 40 80
3. IR register (16-bit) sits between Program ROM and the instruction splitter
4. No D flip-flop in the branch path — AND gate connects directly to OR gate

5. Control Unit output pin names are MemWrite and MemRead (no underscore)
6. W_MUX input 0 is wired to ALU output (r_op tunnel)
7. W_MUX input 1 is wired to Imm tunnel
8. W_MUX input 2 is wired to RAM data output
9. R3_Pointer connects to RAM address — R3 is always the memory pointer
10. Data RAM initialized with 00 01 02 03 04 05 06 07 08 09 at reset

End of Build Guide